ZSCRIPT

The Zscript Book

Write your own chart indicators, from your first plot to order book and options data.

TradeZoro · Education only. Not financial advice.

The Zscript Book. © TradeZoro. All rights reserved.

Education only. Not financial advice. Every script in this book is a teaching sample, not a trading result, and none of it promises profit. Data features depend on your plan.

If you know other chart scripting languages, Zscript will feel familiar.

Contents

Part 1. The language

Getting started 7
Anatomy of a script 10
Bars and series 13
Types and values 17
Decisions and loops 21
Functions 24
Inputs 26
Plots and drawings 28
Alerts 32
Our data 34
Publishing and selling 39
Limits 41
Errors explained 44
Error and warning codes 48
Patterns every scripter uses 53
Build four indicators step by step 57
Glossary 62

Part 3. Cookbook

Simple moving average 65
EMA ribbon 66
Average cross marks 67
RSI with zones 68
MACD with colored histogram 69

Bollinger Bands with shading 70
Keltner channel 71
ATR trailing line 72
Supertrend 73
Session VWAP 74
Stochastic 75
CCI 76
On-balance volume 77
Money flow index 78
Donchian channel 79
Swing high and low labels 80
Previous day high and low 81
Higher timeframe trend 82
Inside bars 83
Engulfing candles 84
Fair value gaps 85
Opening range box 86
Session background 87
Volume spike marks 88
Relative volume 89
Bar range as percent 90
Opening gap marks 91
Label the day's high 92
Info table 93
Gradient volume 94
Trend filter background 95
Bars since last cross 96

Correlation with another symbol 97
Regression line and bands 98
Order book imbalance 99
Nearest book walls 100
Delta columns 101
Cumulative delta 102
Delta against price 103
Big trade count 104
POC and value area 105
GEX levels 106
Above or below the flip 107
Distance to the call wall 108
Option range edges 109
IV rank readout 110
Gamma nodes with GEX walls 111
Cross alert 112
Level alert with a message 113

Part 1. The language

Getting started

Zscript is the small language you use on TradeZoro to build your own chart indicators. You write a few lines, press **Add to chart**, and your idea is drawn on the chart. It is free for everyone: writing, saving and publishing scripts.

If you know other chart scripting languages, Zscript will feel familiar. If you have never written a line of code, that is fine too. This manual starts at zero.

What you can build

- Lines and histograms from price, such as moving averages and momentum.
- Marks on the chart, such as arrows when two lines cross.
- Colored backgrounds, boxes, lines and labels.
- Tables with numbers you care about.
- Alerts that fire when your rule is true.
- Indicators that use data most scripting languages do not have: the order book, order flow, GEX levels and ZoroOptions numbers. See Our data.

Your first script

Open ZoroCharts, open the script editor panel, and choose **Zscript** as the language. Paste this:

```
//@zscript 1
indicator("My first line", overlay=true)

average = ta.sma(close, 20)
plot(average, title="20-bar average", color=color.new(#03c988, 0),
linewidth=2)
```

Press **Add to chart**. A green line appears on the price chart. It is the average of the last 20 closing prices, recalculated on every bar.

Here is what each line does:

1. `//@zscript 1` tells the editor this is a Zscript file, version 1. Always the first line.
2. `indicator(...)` names your script. `overlay=true` draws it on top of the price candles. Leave it out (or use `false`) to get a separate pane under the chart.
3. `average = ta.sma(close, 20)` makes a value called `average`. `ta.sma` is a built-in function: the simple moving average. `close` is the closing price of each bar.
4. `plot(...)` draws the value as a line.

Change something

Edit the number `20` to `50` and press **Add to chart** again. The line gets smoother and slower. That is the whole loop: change, run, look.

Where scripts live

Your scripts are saved in your account, next to your chart layout. You can keep as many as you like. Publishing one to the Script Library is optional and free; see Publishing and selling.

What to read next

- Your first script, line by line explains the shape of every script.
- Bars and series explains the one idea that makes everything else easy.
- The Cookbook has working scripts you can copy and change.

Anatomy of a script

Every script has the same four parts, always in this order.

1. The **header**: `//@zscript 1`.
2. The **declaration**: one `indicator(...)` call.
3. The **inputs**: settings the person using your script can change.
4. The **logic and drawing**: calculations, then `plot` and friends.

```
//@zscript 1
indicator("Anatomy demo", overlay=false)

length = input.int(14, "Length", minval=2)
momentum = ta.rsi(close, length)

plot(momentum, "RSI")
hline(70)
hline(30)
```

Comments

Anything after `//` on a line is a comment. The computer ignores it. Use comments to remind yourself why, not what.

```
// this line is ignored
x = close // so is this part
```

Names

Names of values may use letters, digits and `_`, and cannot start with a digit. Names are case sensitive: `Length` and `length` are two different things. Pick names a stranger could read: `fastLength` beats `a1`.

Lines and blocks

One statement per line. Code that belongs together is indented under its header, with two or four spaces. The indent is how Zscript knows where a block ends.

```
up = close > open
bgcolor(up ? color.new(#03c988, 85) : na)
```

A long line can break after an operator or a comma. Indent the continuation by at least one more level than the line it continues.

Running order

Your script is not run once. It is run **once for every bar** on the chart, from the oldest bar to the newest, and again on every tick of the newest bar. Every line sees the world as it was at that bar. The next chapter shows why this matters.

The declaration

`indicator()` takes a title and a few optional settings.

Setting	What it does
`title`	The name shown on the chart and in the list.
`overlay`	`true` draws on the price chart. `false` (default) uses its own pane.
`max_bars_back`	How far back your script may look. Leave it alone unless you see an error asking for it.

A script has exactly one `indicator()` call, and it must come before any drawing.

Bars and series

One run per bar

A chart is a row of bars. Zscript runs your whole script for each bar. When it is working on bar number 500, `close` means "the close of bar 500". When it moves on to bar 501, `close` now means that bar's close.

A value that changes bar by bar is called a **series**. `open`, `high`, `low`, `close`, `volume` and `time` are series that exist in every script. Anything you calculate from a series is a series too.

Looking back

To see an older value of a series, put a number in square brackets after it. `close[1]` is the close one bar ago, `close[2]` is two bars ago, and so on.

```
//@zscript 1
indicator("Bar change")

change1 = close - close[1]
plot(change1, "Change since last bar", style=plot.style_histogram)
```

On the very first bar there is no bar before it, so `close[1]` has no value. Zscript calls "no value" **na** (not available). Any math with `na` gives `na`, and a plot skips `na` bars. That is by design: no value means no line, not a wrong line.

You can test for it and fix it:

```
prev = nz(close[1], close)   // use close itself when there is no
older bar
isMissing = na(close[1])
```

`nz(x, replacement)` swaps `na` for the replacement (or 0 if you leave it out). `na(x)` is `true` when x is missing.

Remembering across bars with var

An ordinary value is recalculated from scratch on each bar. To keep a number alive from bar to bar, declare it with `var`. A `var` is set once, on the first bar, and keeps whatever you assign to it after that.

```
//@zscript 1
indicator("Count green bars")

var int greens = 0
if close > open
    greens := greens + 1

plot(greens, "Green bars so far")
```

Two things to notice. First, `:=` changes an existing value; a single `=` creates a new one. Second, `greens` never resets, so the plot only goes up.

Realtime bars: var and varip

On historical bars each bar runs once. The newest bar runs again on every tick. A plain `var` is rolled back to its value from the end of the last finished bar before each tick, so history and live look the same. Use `varip` only when you truly need a value to survive between ticks of the same bar, for example to count ticks.

```
//@zscript 1
indicator("Ticks on this bar")

varip int ticks = 0
if barstate.isnew
    ticks := 0
ticks := ticks + 1
plot(ticks)
```

On finished history bars this plots 1, because each bar runs once. On the live bar it counts up.

Series are not arrays

A series is "one value per bar". An **array** is a list you fill and read yourself, on a single bar. Both exist; see Types and values.

Rules of thumb

- Think in terms of "what is true on this bar".
- Use `[n]` to look at the past, never the future. Zscript has no way to read a bar that has not happened yet. That is on purpose, and the same rule holds for order book, order flow, GEX and options numbers: you only see what was known at that bar.

- Use `var` when something must be remembered.

Types and values

Every value has a type. Zscript works out the type for you most of the time, and you can write it when you want to be clear.

Type	Holds	Example
`int`	whole numbers	`14`
`float`	numbers with a decimal part	`0.5`
`bool`	`true` or `false`	`close > open`
`string`	text	`"Buy zone"`
`color`	a color	`#03c988`, `color.red`
`series<T>`	one value of type T per bar	`close`
`array<T>`	a list of values of type T	`array.new<float>()`
`map<K,V>`	a lookup table	`map.new<string, float>()`

Making and changing values

```
//@zscript 1
indicator("Values")

float range1 = high - low        // type written out
mid = (high + low) / 2           // type worked out
bool bigBar = range1 > ta.sma(range1, 20) * 1.5

var float biggest = 0.0
if range1 > biggest
    biggest := range1

plot(mid, "Mid")
plotshape(bigBar, "Big bar", shape.circle, location.abovebar)
```

- `=` creates a value. `:=` changes one that already exists.
- Math: `+ - * / %`. Compare with `== != < <= > >=`. Combine tests with `and`, `or`, `not`.
- Text joins with `+`: `"Range " + str.tostring(range1)`.
- `x ? a : b` gives `a` when `x` is true, else `b`.

Colors

Write a color as `#rrggbb`, use a named one such as `color.green`, or build one with `color.new(base, transparency)` where transparency is 0 (solid) to 100 (invisible). `color.rgb(r, g, b)` builds one from numbers.

Arrays

```
//@zscript 1
indicator("Array demo")

var prices = array.new<float>()
array.push(prices, close)
if array.size(prices) > 5
    array.shift(prices)

plot(array.avg(prices), "Average of the last 5 closes")
```

Maps

```
//@zscript 1
indicator("Map demo")

var counts = map.new<string, int>()
key = close > open ? "up" : "down"
map.put(counts, key, nz(map.get(counts, key)) + 1)
plot(nz(map.get(counts, "up")), "Up bars")
```

Tuples

A function can return several values at once. Take them apart with square brackets.

```
//@zscript 1
indicator("Tuple demo", overlay=true)

[middle, upper, lower] = ta.bb(close, 20, 2.0)
plot(middle)
plot(upper)
plot(lower)
```

Limits to know

Arrays, maps, series and drawings each have a size cap so a script cannot use all your memory. See Limits.

Decisions and loops

if / else

```
//@zscript 1
indicator("Trend color", overlay=true)

fast = ta.ema(close, 9)
slow = ta.ema(close, 21)

lineColor = color.gray
if fast > slow
    lineColor := color.green
else if fast < slow
    lineColor := color.red

plot(fast, color=lineColor, linewidth=2)
```

`if` can also give back a value:

```
label1 = if close > open
    "up"
else
    "down"
```

switch

Pick one branch out of several by value:

```
//@zscript 1
indicator("Pick an average", overlay=true)

kind = input.string("EMA", "Type", options=["SMA", "EMA", "WMA"])
len = input.int(20, "Length")

smaLine = ta.sma(close, len)
wmaLine = ta.wma(close, len)
emaLine = ta.ema(close, len)

line1 = switch kind
    "SMA" => smaLine
    "WMA" => wmaLine
    => emaLine

plot(line1)
```

The lone `=>` is the default branch. We work out all three averages first, on every bar, and let the `switch` only choose. Calling `ta` functions inside a branch would leave their memory with gaps.

for loops

`for` counts from one number to another. Use it to look at the last few bars one by one.

```
//@zscript 1
indicator("Count higher closes")

n = input.int(10, "Look at last", minval=1, maxval=50)
higher = 0
for i = 0 to n - 1
    if close[i] > close[i + 1]
        higher += 1

plot(higher, style=plot.style_columns)
```

`for x in myArray` walks the items of an array.

while

```
n = 0
while n < 10 and close[n] > open[n]
    n += 1
```

Keep loops short

Zscript runs your script on every bar, so a loop that runs 500 times does 500 times the work on every one of thousands of bars. Two safeguards protect you and everyone else:

- A loop has a maximum number of turns per bar, and your script has a maximum amount of work per bar. Hit either and the script stops with a clear message on the line that went too far.
- The first run must finish in 20 seconds.

When you can, use the built-in functions such as `ta.highest` or `ta.sma`: they are faster than the same logic in a loop. More in Limits.

`break` leaves a loop early. `continue` skips to the next turn.

Functions

One-line functions

```
//@zscript 1
indicator("Typical price band", overlay=true)

typical(h, l, c) => (h + l + c) / 3
band(x, pct) => x * (1 + pct / 100)

tp = typical(high, low, close)
plot(tp)
plot(band(tp, 1.0))
plot(band(tp, -1.0))
```

Functions with several lines

The last line is the value the function gives back.

```
//@zscript 1
indicator("Smoothed range", overlay=false)

smoothRange(len) =>
    r = high - low
    avg = ta.sma(r, len)
    avg * 100 / close

plot(smoothRange(14), "Range as % of price")
```

Returning more than one value

```
//@zscript 1
indicator("Two bands", overlay=true)

channel(len) =>
    top = ta.highest(high, len)
    bottom = ta.lowest(low, len)
    [top, bottom]

[t, b] = channel(20)
plot(t)
plot(b)
```

Things to know

- A function must be defined before the line that uses it.
- Functions see the series history of what you pass in: inside `f(x)`, `x[1]` is the previous bar's value of whatever you passed.
- A function that calls `ta.*` functions should be called on every bar, not only inside an `if`. The `ta` functions keep a running history, and skipping bars gives wrong answers. Zscript warns you when it spots this.
- Functions cannot call themselves.

Inputs

Anything you want to be changeable goes in an `input.*` call. Each one adds a control to your script's settings dialog.

Call	Control	Example
`input.int`	whole number box	`input.int(14, "Length", minval=1)`
`input.float`	decimal box	`input.float(2.0, "Multiplier", step=0.1)`
`input.bool`	switch	`input.bool(true, "Show labels")`
`input.string`	text or a pick list	`input.string("EMA", "Type", options=["SMA","EMA"])`
`input.color`	color picker	`input.color(#03c988, "Line color")`
`input.source`	pick a price (close, hl2...)	`input.source(close, "Source")`
`input.timeframe`	timeframe picker	`input.timeframe("60", "Higher timeframe")`

```
//@zscript 1
indicator("Tunable bands", overlay=true)

src    = input.source(close, "Source")
len    = input.int(20, "Length", minval=2, maxval=500,
group="Average")
mult   = input.float(2.0, "Width", minval=0.1, step=0.1,
group="Average")
shade  = input.bool(true, "Shade between bands", group="Look")
tint   = input.color(#00adb5, "Tint", group="Look")

[mid, up, lo] = ta.bb(src, len, mult)
p1 = plot(up, color=tint)
p2 = plot(lo, color=tint)
plot(mid, color=color.gray)
fill(p1, p2, color=shade ? color.new(tint, 90) : na)
```

Good habits

- Give every input a short, plain title and a sensible default. The default is what everyone sees first.
- Set `minval` and `maxval` so nobody can type 0 into a length.
- Use `group="..."` to keep long lists tidy.
- Add `tooltip="..."` to explain anything a newcomer would ask about.
- Inputs are read once per run. You cannot change an input while the script is running.

Plots and drawings

plot

`plot(series, title, color, linewidth, style)` draws one value per bar.

Styles: `plot.style_line` (default), `plot.style_histogram`, `plot.style_columns`, `plot.style_area`, `plot.style_circles`, `plot.style_stepline`, `plot.style_cross`.

```
//@zscript 1
indicator("Volume with color")

up = close >= open
plot(volume, "Volume", color=up ? color.green : color.red,
style=plot.style_columns)
```

A color can differ bar by bar, as above.

hline and fill

`hline(price)` draws a flat line across the pane. `fill(a, b, color)` shades between two plots or two hlines.

```
//@zscript 1
indicator("RSI zone")

r = ta.rsi(close, 14)
top = hline(70)
bot = hline(30)
fill(top, bot, color.new(color.gray, 90))
plot(r, "RSI")
```

Marks on bars

`plotshape(condition, title, shape, location)` puts a small shape on every bar where the condition is true. `plotchar` does the same with a letter or symbol. `plotarrow` draws an arrow whose length follows a number.

```
//@zscript 1
indicator("Cross marks", overlay=true)

fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast)
plot(slow)
plotshape(ta.crossover(fast, slow), "Cross up", shape.triangleup,
location.belowbar, color.green)
plotshape(ta.crossunder(fast, slow), "Cross down",
shape.triangledown, location.abovebar, color.red)
```

Coloring the chart itself

`bgcolor(color)` tints the pane background on a bar. `barcolor(color)` recolors the candle. `plotcandle(o, h, l, c)` draws your own candles.

```
//@zscript 1
indicator("Inside bars", overlay=true)

inside = high < high[1] and low > low[1]
barcolor(inside ? color.yellow : na)
bgcolor(inside ? color.new(color.yellow, 90) : na)
```

Labels, lines, boxes and tables

These are **drawings**. Unlike a plot, they are objects you create, move and delete. Each has a `.new` call and `set_` calls to change it.

```
//@zscript 1
indicator("Swing labels", overlay=true)

ph = ta.pivothigh(high, 5, 5)
pl = ta.pivotlow(low, 5, 5)
if not na(ph)
    label.new(bar_index - 5, ph, "H",
style=label.style_label_down, color=color.red,
textcolor=color.white)
if not na(pl)
    label.new(bar_index - 5, pl, "L", style=label.style_label_up,
color=color.green, textcolor=color.white)
```

```
//@zscript 1
indicator("Today's range box", overlay=true)

var box rangeBox = na
if dayofmonth != dayofmonth[1]
    rangeBox := box.new(bar_index, high, bar_index, low,
border_color=color.gray, bgcolor=color.new(color.gray, 90))
else
    box.set_right(rangeBox, bar_index)
    box.set_top(rangeBox, math.max(box.get_top(rangeBox), high))
    box.set_bottom(rangeBox, math.min(box.get_bottom(rangeBox),
low))
```

```
//@zscript 1
indicator("Info table", overlay=true)

var t = table.new(position.top_right, 2, 2)
if barstate.islast
    table.cell(t, 0, 0, "Close")
    table.cell(t, 1, 0, str.tostring(close, "#.##"))
    table.cell(t, 0, 1, "Volume")
    table.cell(t, 1, 1, str.tostring(volume, "#"))
```

`line.new(x1, y1, x2, y2)` draws a segment. `line.delete(id)` removes it. Each kind has `get_` and `set_` calls, and a `*.delete` call. The reference lists them all.

Drawing limits

Each kind of drawing has a maximum count on screen (labels, lines, boxes, tables). When you go over, the oldest ones are removed first. If you create drawings in a loop, delete the ones you no longer need. See Limits.

Alerts

An alert is a rule that is true or false on each bar. When it turns true, TradeZoro can tell you.

alertcondition

Declare the rule once. It then shows up in the alert dialog on the chart, where you choose how to be told.

```
//@zscript 1
indicator("Cross alerts", overlay=true)

fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast)
plot(slow)

alertcondition(ta.crossover(fast, slow), "Fast crossed above
slow", "9 EMA is above 21 EMA")
alertcondition(ta.crossunder(fast, slow), "Fast crossed below
slow", "9 EMA is below 21 EMA")
```

alert()

`alert(message)` fires from inside your code, which lets you build the message from values.

```
//@zscript 1
indicator("Big bar alert", overlay=true)

big = (high - low) > ta.atr(14) * 2
if big and barstate.isconfirmed
    alert("Big bar: range " + str.tostring(high - low, "#.##"))
```

`barstate.isconfirmed` is true once the bar has closed. Alerting only on closed bars avoids alerts that appear and vanish within a bar.

Good to know

- Alerts run while the chart is open in your browser, using the same data as the chart.
- A message is plain text. Keep it short.
- An alert on data from the order book, order flow, GEX or options follows the same plan rules as showing that data on a chart.

Our data

Most scripting languages stop at price and volume. Zscript can also read the extra data TradeZoro already shows on its charts. Each kind lives in its own namespace.

Namespace	What it is	Plan
`book.*`	Order book (level 2): resting orders above and below price	Pro
`flow.*`	Order flow: trades, delta, big trades, footprint, volume profile	Pro
`gex.*`	GEX+ levels: call wall, put wall, flip, zero gamma	GEX+
`opt.*`	ZoroOptions summary: gamma nodes, ranges, IV rank, max pain	Pro

Your script gets the data your plan gives you on the chart, no more. When your plan does not include a data kind, the call returns `na` and you see one short notice on the chart, once. The script keeps running; the rest of your lines still draw.

No peeking

Every value is the value known **at that bar**. There is no way to read a future bar, so a script that uses this data looks the same live and in Replay. Order book, GEX and options numbers are snapshots: a snapshot is stored at some time and used for the bars that follow it, until the next one arrives. For GEX and ZoroOptions numbers, treat them as "updated a few minutes ago", never as an instant live feed.

Order book: book

```
book.bid(level)        price of the bid at depth `level` (0 = best)
book.ask(level)        price of the ask at depth `level`
book.bidsize(level)    size resting there
book.asksize(level)    size resting there
book.depth(side, n)    total size in the best n levels of a side
(book.side_bid or book.side_ask)
book.imbalance(n)      (bid depth - ask depth) / (bid depth + ask
depth) in the best n levels, -1 to 1
book.wall(side, minSize)  price of the nearest level with at least
minSize resting (side is book.side_bid or book.side_ask)
book.heat(price)       size resting at that price (nearest level
within one tick)
```

Available where we have a depth feed: the futures hub and the supported crypto venues. On other symbols you get `na` and a notice.

```
//@zscript 1
indicator("Book imbalance", overlay=false)

imb = book.imbalance(10)
plot(imb, "Imbalance (10 levels)", style=plot.style_columns,
color=imb >= 0 ? color.green : color.red)
hline(0)
```

Order flow: flow

```
flow.buyvol, flow.sellvol    volume that traded at the ask / at the
bid, per bar
flow.delta                   buyvol - sellvol for the bar
flow.cvd                     running total of delta
flow.trades                  number of trades in the bar
flow.bigtrades(minSize)      count of single trades of at least
minSize
flow.footprint(price)        volume traded at one price level
inside the bar
flow.bigtrades_vol(minSize)  total size of those big trades
flow.poc, flow.vah, flow.val point of control, value area high and
low of the bar
```

```
//@zscript 1
indicator("Delta and CVD")

plot(flow.delta, "Delta", style=plot.style_columns,
color=flow.delta >= 0 ? color.green : color.red)
plot(flow.cvd, "CVD", color=color.white)
```

GEX levels: gex

```
gex.callwall   strike with the largest call gamma
gex.putwall    strike with the largest put gamma
gex.flip       gamma flip level
gex.zero_upper, gex.zero_lower   the two zero-gamma levels around
price
gex.net        net gamma number
gex.level(n)   the nth strongest level (0 = strongest)
gex.updated    when the levels were computed (ms)
```

ES and NQ use the same mapping as the GEX+ page.

```
//@zscript 1
indicator("GEX walls", overlay=true)

plot(gex.callwall, "Call wall", color=color.green,
style=plot.style_stepline)
plot(gex.putwall, "Put wall", color=color.red,
style=plot.style_stepline)
plot(gex.flip, "Flip", color=color.yellow,
style=plot.style_stepline)
```

ZoroOptions: opt

```
opt.node(n)      price of the nth gamma node
opt.range(kind)  a pair [low, high] for kind "day", "week" or
"expected"
opt.ivrank       IV rank, 0 to 100
opt.pc           put/call ratio
opt.maxpain      max pain strike
opt.zone         the current zone label
opt.updated      when the data was computed (ms): show it as
"updated X min ago"
```

```
//@zscript 1
indicator("Option ranges", overlay=true)

[lo, hi] = opt.range("expected")
plot(hi, "Range high", color=color.gray, style=plot.style_stepline)
plot(lo, "Range low", color=color.gray, style=plot.style_stepline)
```

Handling "no data"

Because these calls return `na` when data is missing, protect your math with `nz` or `na()`:

```
//@zscript 1
indicator("Safe delta", overlay=false)

d = nz(flow.delta, 0)
plot(d, "Delta (0 when no data)", style=plot.style_columns)
```

Education only. Data samples in this manual are labelled "Sample" and are not results.

Publishing and selling

Publishing is free for everyone. You can share a script openly, or sell it through Creator Hub.

Publish a script

1. Open your script in the editor and press **Publish**.
2. Give it a name and a short description. Pick a category.
3. Choose how people get it: - **Open source**: anyone who adds it can read the code. - **Protected**: people can use it but not read it.
4. Choose **free** or **paid**.
5. Add a short note about what changed in this version.

The script then appears in the Script Library. Each time you publish again, you add a new version. People who already use your script can update when they choose.

Protected scripts

When you publish a protected script, the source never leaves your account. We compile it, seal the compiled program on our server, and only send it to people who are allowed to use it. Paid and protected scripts run only for people who have it. Nobody else is sent the code.

A protected script runs in the same sandbox as every other script. It cannot reach the network, your storage or the page.

Selling

A paid script is listed in Creator Hub like other creator products. Payouts work the same way as the rest of Creator Hub; see the Creator Hub rules for the payout terms. We do not publish prices in this manual: you set yours in the publish dialog.

Rules for listings

- Describe honestly what the script does.
- Do not claim profit, win rates or results. Education only. Not financial advice.
- Sample screenshots must say "Sample".
- A script that tries to break the sandbox, hide what it does, or harm the viewer is removed.

Data in published scripts

If your script uses book, flow, GEX or options data, people who run it see that data only if their plan includes it, exactly as on a chart. Say so in your description so nobody is surprised.

Limits

Scripts from other members run in your browser. To make that safe, every script runs inside a locked room: it can do math and draw on the chart, and nothing else. It cannot open web connections, read your cookies or storage, or touch the page.

What a script can never do

- Reach the network.
- Read or write your browser storage or cookies.
- Change the page, other than through the plots and drawings it declares.
- Run text as code. Zscript has no `eval`.

The numbers

These are the limits the engine enforces today.

What	Limit
Script size	256 KB, 20,000 lines
Nesting depth	128
Bars in one run	50,000
First full run	20 seconds
Work per bar	1,000,000 operations
Turns of one loop, per bar	100,000
Function calls inside each other	64 (a function cannot call itself)
Items in one array	100,000 (1,000,000 across all arrays)
Keys in one map	50,000
Length of one string	4,096 characters
Plots, marks, fills, hlines, bgcolor and barcolor calls	64 together
Labels, lines, boxes	50 each by default, up to 500 with `max_labels_count` and friends
Tables	up to 20; 1,000 cells each
Alerts kept per run	500
`request.security` calls	16
Look-back with `[n]`	500 bars by default, up to 5,000 with `max_bars_back`

What happens when you hit one

Limit	What you see
Run takes longer than 20 seconds	The run stops (E401).
Too much work on one bar	The script stops on that bar and shows the line (E403).
A loop turns too many times	The loop stops with an error on its line (E404).
Too many items in an array or map	Error on the line that added the extra item (E405, E406).
Too many plots	Error when the script is added (E341).
Too many labels, lines or boxes	The oldest are removed first. This is not an error.
Too many tables	Error (E408).

Every code is explained in Error and warning codes.

Staying under the limits

- Prefer built-ins (`ta.highest` , `ta.sma`) over loops.
- Keep loops short and fixed in length.
- Delete drawings you no longer need.
- Do heavy work only on `barstate.islast` when you only need the result on the newest bar.

Errors explained

When something is wrong, Zscript tells you the **line and column**, says what happened in plain English, and when it can, suggests a fix ("Did you mean ...?").

The three kinds

- **Errors** stop the script from running. The editor underlines the spot in red.
- **Warnings** let it run but point at something risky. Underlined in amber.
- **Notices** are short one-time notes on the chart, such as "Order book data needs Pro".

Common errors

Messages below are written the way the engine words them. Your names and numbers fill the gaps. Every code is listed in Error and warning codes.

E303 Unknown name

```
3:11  E303  Unknown name `smaa`.
            Did you mean `ta.sma`?
```

A typo. Names are case sensitive, and every built-in lives behind its namespace (`ta.` , `math.` , `str.`).

E309 Wrong type for an argument

```
5:8   E309  Argument `title` of `plot` must be string, got float.
```

You passed a number where text goes. Check the order of the arguments, or name them: `plot(x, title="Mine")`.

E201 Missing header

```
1:1   E201  Missing header.
            Put `//@zscript 1` on the first line.
```

Add the header line. Then make sure there is exactly one `indicator(...)` (E301 means it is missing, E302 means there are two).

E105 and E205 Indentation

```
6:1   E105  Indentation does not match any outer block.
```

Indent only under `if`, `for`, `while`, `switch` and function bodies, and line up the lines of one block exactly.

E305 Already declared

```
7:5   E305  `greens` is already declared in this scope.
            Use `:=` to change it.
```

A single `=` makes a new value; `:=` changes one that exists. E306 is the reverse: using `:=` before the value was ever made.

E209 One equals sign in a condition

```
4:8   E209  `=` used in a condition.
            Use `==` to compare, `:=` to reassign.
```

E340 Plot inside a block

```
9:5   E340  `plot` must be called at top level (not inside if/for/
functions).
```

Plots and their friends sit at the top of the script. To plot only sometimes, make the value `na` when you do not want it: `plot(cond ? x : na)`.

W101 A `ta` function that skips bars

```
12:9  W101  `ta.ema` is called only on some bars (inside `if`), so
its state skips the other bars.
            Compute it at top level and use the result inside the
`if`.
```

A warning, not an error. The script runs, but the numbers can be wrong. Do what the hint says.

Runtime errors (E4xx)

```
12:5  E404  Loop ran more than 100,000 times on one bar.
```

Shorten the loop, or use a built-in. See Limits. `E402` on a history look-back means you looked further back than `max_bars_back` allows; raise it in `indicator()`.

No value (na)

An `na` result is not an error. It means "nothing to show on this bar". See Bars and series. Division by zero gives `na`, not an error.

If you are stuck

Cut the script down to the smallest piece that still shows the problem. Add lines back one at a time until the error returns. The line it returns on is the cause.

Error and warning codes

Each message has a code, a line and column, and often a hint. Words in {braces} are filled in with your own names and numbers.

Lexer / header

Code	Meaning
E101	Unexpected character `{c}`.
E102	Unterminated string (closing quote missing).
E103	Bad color literal `{text}` (use #RRGGBB or #RRGGBBAA).
E104	Bad number `{text}`.
E105	Indentation does not match any outer block.
E106	Source is too large (limit 256 KB).
E201	Missing header. Hint: put `//@zscript 1` on the first line.
E202	Unsupported version `{n}`. Hint: use `//@zscript 1`.
E203	Unexpected `{token}`, expected {what}.
E204	Missing closing `)` / `]`.
E205	Expected an indented block after `{kw}`.
E206	`=>` default case must be last in switch.
E207	Nesting too deep (limit 128).
E208	`break` / `continue` outside a loop.
E209	`=` used in a condition. Hint: use `==` to compare, `:=` to reassign.
E210	Reserved word `{w}` cannot be used here.

Checker

Code	Meaning
E301	Missing `indicator(...)` declaration. Hint: add `indicator("My script")` after the header.
E302	`indicator()` declared twice.
E303	Unknown name `{x}`. Hint: Did you mean `{y}`?
E304	Unknown function `{f}`. Hint: Did you mean `{g}`?
E305	`{x}` is already declared in this scope. Hint: use `:=` to change it.
E306	Cannot reassign `{x}` with `:=` before it is declared. Hint: declare it with `=` first.
E307	`{f}` expects {n} arguments, got {m}.
E308	`{f}` has no parameter named `{p}`. Hint: Did you mean `{q}`?
E309	Argument `{p}` of `{f}` must be {type}, got {type2}.
E310	Cannot assign {type} to `{x}` declared as {type2}.
E311	`{x}` is {type}; cannot assign {type2}.
E312	Operator `{op}` cannot be used with {type} and {type2}.
E313	Condition must be bool or number, got {type}.
E314	Tuple has {n} values but {m} names.
E315	`{x}` is not a function.
E316	History `[]` needs a number of bars, got {type}.
E317	Built-in `{x}` cannot be reassigned.
E318	Function `{f}` calls itself (recursion is not allowed).
E319	Functions must be defined at top level.
E320	Method `{m}` does not exist for {type}. Hint: Did you mean `{n}`?
E321	Positional argument after a named one.

Code	Meaning
E322	Function `{f}` declared twice.
E330	Inputs must be called at top level with a literal default.
E340	`{f}` must be called at top level (not inside if/for/functions).
E341	Too many plots (limit 64).
E350	request.security expression uses the script variable `{x}` . Hint: compute it inside the expression, or move it into a function.
E351	lookahead is not allowed.
E352	Timeframe `{tf}` is lower than the chart timeframe.
E353	Too many request.security calls (limit 16).
W101	`{f}` is called only on some bars (inside `if`), so its state skips the other bars. Hint: compute it at top level and use the result inside the `if` .
W102	`{f}` inside a loop shares one state for every iteration.
W103	Variable `{x}` is never used.

Runtime

Code	Meaning
E401	Script took longer than 20 s.
E402	History `[{n}]` is beyond max_bars_back ({m}). Hint: raise max_bars_back in indicator().
E403	Too many operations on one bar (limit 1,000,000). Hint: a loop may be too large.
E404	Loop ran more than 100,000 times on one bar.
E405	Array too large (limit 100,000 items) / too many items in all arrays (1,000,000).
E406	Map too large (limit 50,000 keys).
E407	String too long (limit 4,096).
E408	Too many {kind} drawings is not an error: oldest is removed. (Tables over 20 -> E408.)
E409	Division by zero is NOT an error: result is na.
E410	Index {i} out of range for array of size {n}.
E411	Called a method on na (`{what}`).
E412	`{f}` length must be a positive number, got {v}.
E413	Too many bars (limit 50,000).
E414	Function calls nested too deep (limit 64).
E415	request.security data for {sym} {tf} is not available (warning-level notice, value na).
E499	Internal engine error (report this script to support).

Patterns every scripter uses

Once you know the pieces, a few patterns turn up again and again. Learn these and most scripts write themselves.

1. Compute first, decide after

Work out every indicator at the top of the script, on every bar. Then use the finished numbers inside `if` and `switch`. This keeps each `ta` function's memory complete.

```
//@zscript 1
indicator("Compute first", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
trendUp = fast > slow

plot(trendUp ? fast : slow, "Active line", color=trendUp ?
color.green : color.red)
```

2. Use `na` to mean "draw nothing"

A plot cannot sit inside an `if`. Instead make the value `na` on the bars where you want nothing.

```
//@zscript 1
indicator("Only when quiet", overlay=false)
r = (high - low) / close * 100
quiet = r < ta.sma(r, 20) * 0.7
plot(quiet ? r : na, "Quiet bars", style=plot.style_columns,
color=color.gray)
```

3. A flag that flips

Remember the last state with `var` and change it only when something happens.

```
//@zscript 1
indicator("Last cross direction", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
var int dir = 0
if ta.crossover(fast, slow)
    dir := 1
if ta.crossunder(fast, slow)
    dir := -1
bgcolor(dir == 1 ? color.new(color.green, 94) : dir == -1 ?
color.new(color.red, 94) : na)
```

4. Remember a price at the signal

`ta.valuewhen` hands you the value a series had when a condition was last true.

```
//@zscript 1
indicator("Price at last cross", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
lastCrossPrice = ta.valuewhen(ta.cross(fast, slow), close, 0)
plot(lastCrossPrice, "Close at last cross", color=color.yellow,
style=plot.style_stepline)
```

5. Normalize so scripts work on any symbol

Divide by price or by ATR so your numbers mean the same on every market.

```
//@zscript 1
indicator("Move in ATRs")
move = (close - close[10]) / ta.atr(14)
hline(0)
plot(move, "10-bar move in ATRs", style=plot.style_histogram,
color=move >= 0 ? color.green : color.red)
```

6. Smooth a noisy value

```
//@zscript 1
indicator("Smoothed volume")
plot(volume, "Volume", style=plot.style_columns,
color=color.new(color.gray, 60))
plot(ta.ema(volume, 10), "Smoothed", color=color.orange)
```

7. Limit how many drawings you make

Create a drawing only when it is new, and delete old ones.

```
//@zscript 1
indicator("One line for the latest high", overlay=true)
var line hiLine = na
ph = ta.pivothigh(high, 5, 5)
if not na(ph)
    line.delete(hiLine)
    hiLine := line.new(bar_index - 5, ph, bar_index, ph,
extend=extend.right, color=color.red)
```

8. Draw the summary only on the last bar

```
//@zscript 1
indicator("Last bar summary", overlay=true)
upBars = ta.sma(close > open ? 1.0 : 0.0, 50) * 100
if barstate.islast
    label.new(bar_index, high, "Up bars: " + str.tostring(upBars,
"#") + "%", style=label.style_label_down)
```

9. Reuse with a function

If you write the same three lines twice, make a function.

```
//@zscript 1
indicator("Two stochastics")
stoch1(len) => ta.stoch(close, high, low, len)
plot(stoch1(14), "Slow", color=color.blue)
plot(stoch1(5), "Fast", color=color.orange)
```

10. Guard data calls

Every call into `book`, `flow`, `gex` and `opt` can return `na`. Use `nz` when a missing value should count as zero, and `na()` when it should mean "skip".

```
//@zscript 1
indicator("Guarded delta")
d = flow.delta
plot(na(d) ? na : d, "Delta", style=plot.style_columns,
color=nz(d) >= 0 ? color.green : color.red)
```

Build four indicators step by step

The best way to learn is to build. Here are four indicators, from small to bigger. Type each step in and watch it grow.

Project 1. A trend ribbon

Goal: a line that is green in an up-trend and red in a down-trend.

Step 1. Draw the average.

```
//@zscript 1
indicator("Ribbon, step 1", overlay=true)
mid = ta.ema(close, 21)
plot(mid)
```

Step 2. Decide the trend: price above the average means up.

```
//@zscript 1
indicator("Ribbon, step 2", overlay=true)
mid = ta.ema(close, 21)
up = close > mid
plot(mid, color=up ? color.green : color.red, linewidth=2)
```

Step 3. Make the length a setting and add a soft band using ATR.

```
//@zscript 1
indicator("Trend ribbon", overlay=true)
len = input.int(21, "Length", minval=2)
width = input.float(1.0, "Band width in ATRs", minval=0.1,
step=0.1)
mid = ta.ema(close, len)
rng = ta.atr(14) * width
up = close > mid
tone = up ? color.green : color.red
p1 = plot(mid + rng, "Upper", color=color.new(tone, 80))
p2 = plot(mid - rng, "Lower", color=color.new(tone, 80))
plot(mid, "Middle", color=tone, linewidth=2)
fill(p1, p2, color.new(tone, 92))
```

You just used inputs, a condition, a color choice and a fill. That is most of the language.

Project 2. Previous day levels

Goal: draw yesterday's high, low and close as flat lines on a small chart.

Step 1. Ask for the daily values. `[1]` means the last finished day.

```
//@zscript 1
indicator("Prior day, step 1", overlay=true)
pdh = request.security(syminfo.tickerid, "D", high[1])
plot(pdh, "Prior day high", color=color.green,
style=plot.style_stepline)
```

Step 2. Add the low and the close, and label the newest bar.

```
//@zscript 1
indicator("Prior day levels", overlay=true)
pdh = request.security(syminfo.tickerid, "D", high[1])
pdl = request.security(syminfo.tickerid, "D", low[1])
pdc = request.security(syminfo.tickerid, "D", close[1])
plot(pdh, "Prior day high", color=color.green,
style=plot.style_stepline)
plot(pdl, "Prior day low", color=color.red,
style=plot.style_stepline)
plot(pdc, "Prior day close", color=color.gray,
style=plot.style_stepline)

if barstate.islast
    label.new(bar_index, pdh, "PDH", style=label.style_label_left,
color=color.green, textcolor=color.white)
    label.new(bar_index, pdl, "PDL", style=label.style_label_left,
color=color.red, textcolor=color.white)
```

Project 3. A volume pressure panel

Goal: a pane that shows how much of each bar's volume was at the ask, with a smoothed line. Needs order flow data, which follows your plan.

Step 1. The share of buying volume, per bar.

```
//@zscript 1
indicator("Buy share, step 1")
share = flow.buyvol / (flow.buyvol + flow.sellvol) * 100
hline(50)
plot(share, "Buy share %", style=plot.style_columns,
color=color.gray)
```

Step 2. Color by side and add a smooth line.

```
//@zscript 1
indicator("Buy share")
share = flow.buyvol / (flow.buyvol + flow.sellvol) * 100
hline(50)
plot(share, "Buy share %", style=plot.style_columns, color=share
>= 50 ? color.new(color.green, 40) : color.new(color.red, 40))
plot(ta.ema(share, 10), "Smoothed", color=color.white, linewidth=2)
```

When the plan lacks order flow data, `share` is `na`, nothing draws, and you see a short notice. The script itself is fine.

Project 4. A level board with alerts

Goal: show GEX levels as lines, a small table with the distance to each, and an alert when price crosses the flip level.

```
//@zscript 1
indicator("Level board", overlay=true)

plot(gex.callwall, "Call wall", color=color.green,
style=plot.style_stepline)
plot(gex.putwall, "Put wall", color=color.red,
style=plot.style_stepline)
plot(gex.flip, "Flip", color=color.yellow,
style=plot.style_stepline)

var t = table.new(position.top_right, 2, 3)
if barstate.islast
    table.cell(t, 0, 0, "To call wall")
    table.cell(t, 1, 0, str.tostring(gex.callwall - close, "#.##"))
    table.cell(t, 0, 1, "To put wall")
    table.cell(t, 1, 1, str.tostring(close - gex.putwall, "#.##"))
    table.cell(t, 0, 2, "To flip")
    table.cell(t, 1, 2, str.tostring(close - gex.flip, "#.##"))

alertcondition(ta.crossover(close, gex.flip), "Crossed above
flip", "Price crossed above the flip level")
alertcondition(ta.crossunder(close, gex.flip), "Crossed below
flip", "Price crossed below the flip level")
```

GEX levels are updated a few minutes at a time, so this board shows the latest update, not an instant feed. Education only. Not financial advice.

Where to go from here

Open the Cookbook and change a recipe. Then try your own idea. When it works, think about publishing it.

Glossary

Alert. A rule that tells you when it becomes true.

Array. A list of values you fill and read yourself.

ATR. Average true range: the typical size of a bar.

Bar. One candle: its open, high, low, close and volume.

Bar index. The number of a bar, counting from 0 at the oldest.

Bool. A value that is `true` or `false`.

Built-in. A function or value that comes with Zscript, such as `ta.sma` or `close`.

CVD. Cumulative volume delta: the running total of delta.

Declaration. The `indicator(...)` line that names the script.

Delta. Volume traded at the ask minus volume traded at the bid.

Drawing. A label, line, box or table that you create and can change or delete.

Footprint. Volume traded at each price level inside a bar.

Function. A named piece of logic, such as `ta.sma` or one you write yourself.

GEX. Gamma exposure: a way of summarising option positions into price levels.

Header. The first line, `//@zscript 1`.

History. Older values of a series, read with `[n]`.

Input. A setting the person using the script can change.

Level two (L2). The order book: resting buy and sell orders at each price.

na. "Not available": no value on this bar.

Namespace. A group of built-ins sharing a prefix, such as `ta.` or `book.`.

Overlay. Drawing on the price chart instead of in a separate pane.

Pane. A chart area. Price is one pane; an indicator with `overlay=false` gets its own.

Pivot. A swing high or low, confirmed by bars on both sides.

Plot. A line, histogram or other shape with one value per bar.

Plan. Your TradeZoro plan, which decides which data kinds your scripts can read.

Protected script. A published script that people can use but not read.

Series. A value that has one number per bar and keeps its history.

Tuple. Several values returned together, taken apart with `[a, b] = f()`.

var. A keyword that makes a value remember itself from bar to bar.

ZoroOptions. TradeZoro's options summary: gamma nodes, ranges, IV rank and more.

Part 3. Cookbook

Simple moving average

The classic smooth line.

```
//@zscript 1
indicator("SMA", overlay=true)
len = input.int(20, "Length", minval=1)
plot(ta.sma(close, len), "SMA", color=#03c988, linewidth=2)
```

How it works

The average of the last `len` closes. Longer length, smoother and slower line. Try: switch `close` for `hl2` to average the middle of each bar instead.

EMA ribbon

Four averages that fan out in a trend.

```
//@zscript 1
indicator("EMA ribbon", overlay=true)
plot(ta.ema(close, 8), "8", color=color.new(#03c988, 0))
plot(ta.ema(close, 13), "13", color=color.new(#03c988, 25))
plot(ta.ema(close, 21), "21", color=color.new(#03c988, 50))
plot(ta.ema(close, 34), "34", color=color.new(#03c988, 70))
```

How it works

Each line is an exponential average. When they spread apart, price has been moving one way for a while. When they tangle, it has not. The fading color helps you tell the lines apart.

Average cross marks

Arrows when a fast line crosses a slow one.

```
//@zscript 1
indicator("Cross marks", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast, "Fast", color=color.green)
plot(slow, "Slow", color=color.red)
plotshape(ta.crossover(fast, slow), "Up", shape.triangleup,
location.belowbar, color.green)
plotshape(ta.crossunder(fast, slow), "Down", shape.triangledown,
location.abovebar, color.red)
```

How it works

`ta.crossover(a, b)` is true on the bar where `a` moves from below `b` to above it. This is a teaching sample of a rule, not a trading system.

RSI with zones

RSI with shaded 30 and 70 lines.

```
//@zscript 1
indicator("RSI zones")
len = input.int(14, "Length", minval=2)
r = ta.rsi(close, len)
top = hline(70)
bot = hline(30)
fill(top, bot, color.new(color.gray, 92))
plot(r, "RSI", color=#00adb5)
```

How it works

RSI measures how strong recent up moves are compared with down moves, on a 0 to 100 scale. 70 and 30 are common reference lines, not signals on their own.

MACD with colored histogram

Momentum from two averages.

```
//@zscript 1
indicator("MACD")
[macdLine, signal, hist] = ta.macd(close, 12, 26, 9)
plot(hist, "Histogram", style=plot.style_columns, color=hist >=
0 ? color.green : color.red)
plot(macdLine, "MACD", color=color.white)
plot(signal, "Signal", color=color.yellow)
```

How it works

The histogram is the gap between the MACD line and its signal line. Green above zero, red below.

Bollinger Bands with shading

An average and a band of standard deviations.

```
//@zscript 1
indicator("Bands", overlay=true)
len = input.int(20, "Length")
mult = input.float(2.0, "Width", step=0.1)
[mid, up, lo] = ta.bb(close, len, mult)
p1 = plot(up, "Upper", color=#00adb5)
p2 = plot(lo, "Lower", color=#00adb5)
plot(mid, "Middle", color=color.gray)
fill(p1, p2, color.new(#00adb5, 92))
```

How it works

`ta.bb` returns three values at once, so we take them apart with square brackets.

Keltner channel

An average with bands from average true range.

```
//@zscript 1
indicator("Keltner", overlay=true)
[mid, up, lo] = ta.kc(close, 20, 1.5)
p1 = plot(up, "Upper", color=color.orange)
p2 = plot(lo, "Lower", color=color.orange)
plot(mid, "Middle", color=color.gray)
fill(p1, p2, color.new(color.orange, 93))
```

How it works

Same idea as Bollinger Bands, but the width follows the typical bar size (ATR) rather than standard deviation.

ATR trailing line

A line that keeps its distance from price.

```
//@zscript 1
indicator("ATR trail", overlay=true)
mult = input.float(3.0, "ATR multiple", step=0.5)
atr1 = ta.atr(14)
var float trail = na
raw = close - atr1 * mult
trail := na(trail) ? raw : math.max(trail, raw)
plot(trail, "Trail", color=color.orange, style=plot.style_stepline)
```

How it works

The line only moves up, never down, so it traces a path that stays a set number of bar-sizes below the close. A teaching example of `var`.

Supertrend

A built-in trend line that flips color.

```
//@zscript 1
indicator("Supertrend", overlay=true)
[st, dir] = ta.supertrend(3.0, 10)
plot(st, "Supertrend", color=dir < 0 ? color.green : color.red,
linewidth=2)
```

How it works

`dir` is -1 in an up-trend and 1 in a down-trend. We use it to choose the color.

Session VWAP

Average price weighted by volume.

```
//@zscript 1
indicator("VWAP", overlay=true)
plot(ta.vwap(hlc3), "VWAP", color=#03c988, linewidth=2)
```

How it works

`ta.vwap` weights each bar's typical price by its volume and starts again each day (UTC). Pass `anchor="W"` or `"M"` for a weekly or monthly start.

Stochastic

Where the close sits inside the recent range.

```
//@zscript 1
indicator("Stochastic")
k = ta.stoch(close, high, low, 14)
d = ta.sma(k, 3)
hline(80)
hline(20)
plot(k, "%K", color=color.blue)
plot(d, "%D", color=color.orange)
```

How it works

100 means the close is at the top of the last 14 bars' range, 0 the bottom.

CCI

How far typical price strays from its average.

```
//@zscript 1
indicator("CCI")
c = ta.cci(hlc3, 20)
hline(100)
hline(-100)
hline(0)
plot(c, "CCI", color=#00adb5)
```

How it works

Bigger than 100 or smaller than -100 means an unusually big move compared with recent bars.

On-balance volume

A running volume total that follows direction.

```
//@zscript 1
indicator("OBV")
plot(ta.obv, "OBV", color=#03c988)
```

How it works

Volume is added on up closes and subtracted on down closes. Look at the shape, not the number.

Money flow index

RSI that includes volume.

```
//@zscript 1
indicator("MFI")
hline(80)
hline(20)
plot(ta.mfi(hlc3, 14), "MFI", color=color.purple)
```

How it works

Like RSI, but each bar counts more when volume was high.

Donchian channel

Highest high and lowest low of N bars.

```
//@zscript 1
indicator("Donchian", overlay=true)
n = input.int(20, "Length", minval=2)
hi = ta.highest(high, n)
lo = ta.lowest(low, n)
p1 = plot(hi, "High", color=color.green)
p2 = plot(lo, "Low", color=color.red)
fill(p1, p2, color.new(color.gray, 94))
```

How it works

A breakout channel: a close above the top line is a new N-bar high.

Swing high and low labels

Mark pivots after they are confirmed.

```
//@zscript 1
indicator("Pivots", overlay=true)
n = input.int(5, "Bars each side", minval=1, maxval=20)
ph = ta.pivothigh(high, n, n)
pl = ta.pivotlow(low, n, n)
if not na(ph)
    label.new(bar_index - n, ph, "H",
style=label.style_label_down, color=color.red,
textcolor=color.white)
if not na(pl)
    label.new(bar_index - n, pl, "L", style=label.style_label_up,
color=color.green, textcolor=color.white)
```

How it works

A pivot needs `n` bars on each side to be sure, so it is only known `n` bars later. That is why the label is placed at `bar_index - n`: it goes back to where the pivot really was, without ever using a future bar.

Previous day high and low

Lines from yesterday's range.

```
//@zscript 1
indicator("Prior day", overlay=true)
pdh = request.security(syminfo.tickerid, "D", high[1])
pdl = request.security(syminfo.tickerid, "D", low[1])
plot(pdh, "Prior high", color=color.green,
style=plot.style_stepline)
plot(pdl, "Prior low", color=color.red, style=plot.style_stepline)
```

How it works

`request.security` asks for a value from another timeframe. `[1]` on the daily series means "the last finished day", which keeps it free of look-ahead.

Higher timeframe trend

Show the 1-hour trend on a small chart.

```
//@zscript 1
indicator("HTF trend", overlay=true)
tf = input.timeframe("60", "Higher timeframe")
htf = request.security(syminfo.tickerid, tf, ta.ema(close, 50)[1])
plot(htf, "HTF EMA", color=color.yellow, style=plot.style_stepline)
bgcolor(close > htf ? color.new(color.green, 94) :
color.new(color.red, 94))
```

How it works

The background turns green when price is above the higher timeframe average.

Inside bars

Highlight bars within the previous bar's range.

```
//@zscript 1
indicator("Inside bar", overlay=true)
inside = high <= high[1] and low >= low[1]
barcolor(inside ? color.yellow : na)
```

How it works

Returning `na` for a color leaves the candle alone.

Engulfing candles

Mark bars that wrap the previous body.

```
//@zscript 1
indicator("Engulfing", overlay=true)
bull = close > open and close[1] < open[1] and close >= open[1]
and open <= close[1]
bear = close < open and close[1] > open[1] and close <= open[1]
and open >= close[1]
plotshape(bull, "Bullish", shape.arrowup, location.belowbar,
color.green)
plotshape(bear, "Bearish", shape.arrowdown, location.abovebar,
color.red)
```

How it works

A pattern finder in four lines. A pattern is a shape, not a prediction.

Fair value gaps

Box the gap between candle 1 and candle 3.

```
//@zscript 1
indicator("FVG", overlay=true, max_bars_back=5)
bullGap = low > high[2]
bearGap = high < low[2]
if bullGap
    box.new(bar_index - 2, low, bar_index + 10, high[2],
border_color=color.green, bgcolor=color.new(color.green, 88))
if bearGap
    box.new(bar_index - 2, low[2], bar_index + 10, high,
border_color=color.red, bgcolor=color.new(color.red, 88))
```

How it works

A gap exists when the wicks of candles one and three do not overlap. We draw a box from the gap forward ten bars.

Opening range box

Box the first bars of the session.

```
//@zscript 1
indicator("Opening range", overlay=true)
bars = input.int(6, "Bars in range", minval=1)
newDay = dayofmonth != dayofmonth[1]
var int count = 0
var float hi = na
var float lo = na
var box b = na
if newDay
    count := 1
    hi := high
    lo := low
    b := box.new(bar_index, high, bar_index, low,
border_color=color.gray, bgcolor=color.new(color.gray, 90))
else if count < bars
    count += 1
    hi := math.max(hi, high)
    lo := math.min(lo, low)
    box.set_right(b, bar_index)
    box.set_top(b, hi)
    box.set_bottom(b, lo)
```

How it works

Each new day starts a fresh box. For the first `bars` bars it grows to include every high and low.

Session background

Tint the background by time of day (UTC).

```
//@zscript 1
indicator("Session tint", overlay=true)
minuteOfDay = hour * 60 + minute
inSession = minuteOfDay >= 13 * 60 + 30 and minuteOfDay < 20 * 60
bgcolor(inSession ? color.new(color.blue, 94) : na)
```

How it works

`hour` and `minute` are in UTC in Zscript v1, so the window above (13:30 to 20:00 UTC) is the New York cash session in summer. Move the two numbers for other markets or for winter time.

Volume spike marks

Mark bars with unusually high volume.

```
//@zscript 1
indicator("Volume spikes")
mult = input.float(2.0, "Times the average", step=0.1)
avg = ta.sma(volume, 20)
spike = volume > avg * mult
plot(volume, "Volume", style=plot.style_columns, color=spike ?
color.orange : color.gray)
plot(avg, "Average", color=color.white)
```

How it works

Orange columns are bars where volume was more than `mult` times its 20-bar average.

Relative volume

Volume as a multiple of normal.

```
//@zscript 1
indicator("Relative volume")
rv = volume / ta.sma(volume, 30)
hline(1)
plot(rv, "Relative volume", style=plot.style_columns, color=rv >
1 ? #03c988 : color.gray)
```

How it works

1.0 is normal. 2.0 means twice the 30-bar average.

Bar range as percent

How big is each bar, relative to price?

```
//@zscript 1
indicator("Range %")
pct = (high - low) / close * 100
plot(pct, "Range %", style=plot.style_columns, color=#00adb5)
plot(ta.sma(pct, 20), "Average", color=color.white)
```

How it works

Useful for comparing quiet and busy markets on the same scale.

Opening gap marks

Mark days that open away from the prior close.

```
//@zscript 1
indicator("Gap marks", overlay=true)
minGap = input.float(0.3, "Minimum gap %", step=0.1)
newDay = dayofmonth != dayofmonth[1]
gapPct = (open - close[1]) / close[1] * 100
plotshape(newDay and gapPct > minGap, "Gap up", shape.triangleup,
location.belowbar, color.green)
plotshape(newDay and gapPct < -minGap, "Gap down",
shape.triangledown, location.abovebar, color.red)
```

How it works

Compares each day's first open with the last close before it.

Label the day's high

Write the high of the day on the newest bar.

```
//@zscript 1
indicator("Day high label", overlay=true)
var float dayHigh = na
if dayofmonth != dayofmonth[1]
    dayHigh := high
else
    dayHigh := math.max(dayHigh, high)
if barstate.islast
    label.new(bar_index, dayHigh, "Day high " +
str.tostring(dayHigh, "#.##"), style=label.style_label_left)
```

How it works

Drawing only on `barstate.islast` keeps the script light: one label, not thousands.

Info table

A small table with key numbers.

```
//@zscript 1
indicator("Info table", overlay=true)
var t = table.new(position.top_right, 2, 3)
if barstate.islast
    table.cell(t, 0, 0, "ATR(14)")
    table.cell(t, 1, 0, str.tostring(ta.atr(14), "#.##"))
    table.cell(t, 0, 1, "RSI(14)")
    table.cell(t, 1, 1, str.tostring(ta.rsi(close, 14), "#.#"))
    table.cell(t, 0, 2, "Volume")
    table.cell(t, 1, 2, str.tostring(volume, "#"))
```

How it works

The indicator functions are called on every bar (as they must be) and the table is only written on the last bar.

Gradient volume

Color by how big a value is.

```
//@zscript 1
indicator("Gradient")
r = ta.rsi(close, 14)
c = color.from_gradient(r, 30, 70, color.red, color.green)
plot(r, "RSI", style=plot.style_columns, color=c)
```

How it works

`color.from_gradient(value, low, high, colorLow, colorHigh)` blends between two colors.

Trend filter background

Green when above a long average, red when below.

```
//@zscript 1
indicator("Trend filter", overlay=true)
len = input.int(100, "Length")
a = ta.sma(close, len)
plot(a, "Average", color=color.gray)
bgcolor(close > a ? color.new(color.green, 94) :
color.new(color.red, 94))
```

How it works

The simplest possible regime filter.

Bars since last cross

Count bars since a signal.

```
//@zscript 1
indicator("Bars since cross")
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
n = ta.barssince(ta.cross(fast, slow))
plot(n, "Bars since last cross", style=plot.style_columns,
color=#00adb5)
```

How it works

`ta.barssince(condition)` counts bars since the condition was last true.

Correlation with another symbol

How closely two markets move together.

```
//@zscript 1
indicator("Correlation")
other = input.string("NQ1!", "Other symbol")
o = request.security(other, timeframe.period, close)
c = ta.correlation(close, o, 50)
hline(0)
plot(c, "Correlation (50 bars)", color=#00adb5)
```

How it works

Near 1 they move together, near -1 they move opposite, near 0 they do not relate.

Regression line and bands

A fitted line through recent closes.

```
//@zscript 1
indicator("Regression", overlay=true)
len = input.int(50, "Length")
lr = ta.linreg(close, len, 0)
sd = ta.stdev(close, len)
p1 = plot(lr + 2 * sd, "Upper", color=color.gray)
p2 = plot(lr - 2 * sd, "Lower", color=color.gray)
plot(lr, "Line", color=#03c988)
fill(p1, p2, color.new(color.gray, 94))
```

How it works

The fitted line and a band two standard deviations away.

Order book imbalance

Which side has more resting size (Pro data).

```
//@zscript 1
indicator("Book imbalance")
n = input.int(10, "Levels", minval=1, maxval=40)
imb = book.imbalance(n)
hline(0)
plot(imb, "Imbalance", style=plot.style_columns, color=imb >= 0 ?
color.green : color.red)
```

How it works

Above zero means more size resting on the bid side. Without a plan that includes the data, or on a symbol with no depth feed, this shows nothing and a one-time notice explains why.

Nearest book walls

Draw the nearest big resting levels (Pro data).

```
//@zscript 1
indicator("Book walls", overlay=true)
minSize = input.int(500, "Minimum size", minval=1)
plot(book.wall(book.side_bid, minSize), "Bid wall",
color=color.green, style=plot.style_stepline)
plot(book.wall(book.side_ask, minSize), "Ask wall",
color=color.red, style=plot.style_stepline)
```

How it works

A wall is a single price level with at least `minSize` resting. Walls can be pulled at any time, so treat them as information, not a promise.

Delta columns

Buyers minus sellers per bar (Pro data).

```
//@zscript 1
indicator("Delta")
hline(0)
plot(flow.delta, "Delta", style=plot.style_columns,
color=flow.delta >= 0 ? color.green : color.red)
```

How it works

Positive when more volume traded at the ask than at the bid.

Cumulative delta

Running total of delta (Pro data).

```
//@zscript 1
indicator("CVD")
plot(flow.cvd, "CVD", color=#00adb5)
```

How it works

Compare its slope with price to see whether buying or selling is driving the move.

Delta against price

Mark bars where price and delta disagree (Pro data).

```
//@zscript 1
indicator("Delta divergence", overlay=true)
newHigh = high >= ta.highest(high, 20)
weak = flow.delta < 0
plotshape(newHigh and weak, "High on negative
delta", shape.xcross, location.abovebar, color.orange)
newLow = low <= ta.lowest(low, 20)
strong = flow.delta > 0
plotshape(newLow and strong, "Low on positive
delta", shape.xcross, location.belowbar, color.orange)
```

How it works

A new 20-bar high that traded with negative delta (or a low with positive delta) is flagged. It is a thing to look at, not a signal.

Big trade count

How many large single trades hit the bar (Pro data).

```
//@zscript 1
indicator("Big trades")
minSize = input.int(50, "Minimum trade size", minval=1)
n = flow.bigtrades(minSize)
plot(n, "Big trades", style=plot.style_columns, color=n > 0 ?
color.orange : color.gray)
```

How it works

Counts single trades of at least `minSize` contracts or units.

POC and value area

Point of control with value area edges (Pro data).

```
//@zscript 1
indicator("POC and value area", overlay=true)
plot(flow.poc, "POC", color=color.yellow,
style=plot.style_stepline)
p1 = plot(flow.vah, "VAH", color=color.gray,
style=plot.style_stepline)
p2 = plot(flow.val, "VAL", color=color.gray,
style=plot.style_stepline)
fill(p1, p2, color.new(color.gray, 94))
```

How it works

The price with the most volume in the bar, and the range holding most of the volume.

GEX levels

Call wall, put wall and flip as lines (GEX+ data).

```
//@zscript 1
indicator("GEX levels", overlay=true)
plot(gex.callwall, "Call wall", color=color.green,
style=plot.style_stepline)
plot(gex.putwall, "Put wall", color=color.red,
style=plot.style_stepline)
plot(gex.flip, "Flip", color=color.yellow,
style=plot.style_stepline)
```

How it works

The levels come from the latest GEX+ update, not an instant feed. For ES and NQ they follow the GEX+ page mapping.

Above or below the flip

Tint the background by gamma regime (GEX+ data).

```
//@zscript 1
indicator("Flip regime", overlay=true)
f = gex.flip
bgcolor(na(f) ? na : close > f ? color.new(color.green, 94) :
color.new(color.red, 94))
```

How it works

Green above the flip level, red below. No tint when there is no data.

Distance to the call wall

How far price is from a level, in percent (GEX+ data).

```
//@zscript 1
indicator("Distance to call wall")
d = (gex.callwall - close) / close * 100
hline(0)
plot(d, "% to call wall", color=#00adb5)
```

How it works

Positive when price is below the wall.

Option range edges

The expected range as two lines (ZoroOptions data).

```
//@zscript 1
indicator("Option range", overlay=true)
[lo, hi] = opt.range("expected")
plot(hi, "Upper", color=color.gray, style=plot.style_stepline)
plot(lo, "Lower", color=color.gray, style=plot.style_stepline)
```

How it works

Range numbers are updated a few minutes at a time, not tick by tick.

IV rank readout

A table with IV rank and put/call ratio (ZoroOptions data).

```
//@zscript 1
indicator("Options readout", overlay=true)
var t = table.new(position.top_right, 2, 2)
if barstate.islast
    table.cell(t, 0, 0, "IV rank")
    table.cell(t, 1, 0, str.tostring(opt.ivrank, "#.#"))
    table.cell(t, 0, 1, "Put/call")
    table.cell(t, 1, 1, str.tostring(opt.pc, "#.##"))
```

How it works

Shows "NaN" style blanks when your plan lacks the data; the script still runs.

Gamma nodes with GEX walls

Combine two data kinds in one script (GEX+ and ZoroOptions data).

```
//@zscript 1
indicator("Nodes and walls", overlay=true)
plot(opt.node(1), "Node 1", color=color.purple,
style=plot.style_stepline)
plot(opt.node(2), "Node 2", color=color.purple,
style=plot.style_stepline)
plot(gex.callwall, "Call wall", color=color.green,
style=plot.style_stepline)
plot(gex.putwall, "Put wall", color=color.red,
style=plot.style_stepline)
```

How it works

Each data kind follows your plan on its own. If one is missing, only its lines go blank.

Cross alert

An alert you can set from the chart.

```
//@zscript 1
indicator("Cross alert", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast)
plot(slow)
alertcondition(ta.crossover(fast, slow), "Up cross", "Fast EMA
crossed above slow EMA")
alertcondition(ta.crossunder(fast, slow), "Down cross", "Fast EMA
crossed below slow EMA")
```

How it works

After you add the script, the alert dialog lists both rules.

Level alert with a message

Build the message from live values.

```
//@zscript 1
indicator("Level alert", overlay=true)
level = input.float(100.0, "Level")
hline(level)
if ta.crossover(close, level) and barstate.isconfirmed
    alert("Close above " + str.tostring(level, "#.##"))
```

How it works

The message is built from the number you chose in the settings.